



Inżynieria Oprogramowania

dr Katarzyna Grzesiak-Kopeć





06

Metodyki zwinne

AGILE zwinny, zręczny, zwrotny, ruchliwy



**Ogólny pomysł na tworzenie
oprogramowania**

Wiele różnych metod



AGILE zwinny, zręczny, zwrotny, ruchliwy



<https://www.agilealliance.org>

Since 2001, Agile Alliance has been informing and inspiring people and organizations as they explore, apply and expand the values, principles, and practices outlined in the Agile Manifesto.

Read the [Agile Manifesto](#), and learn more about [its history](#), and [its authors](#).



Manifest 2001 workshop in Snowbird, Utah, USA

Manifesto for Agile Software Development

We are uncovering better ways of developing software by doing it and helping others do it.
Through this work we have come to value:

Individuals and interactions over processes and tools

Working software over comprehensive documentation

Customer collaboration over contract negotiation

Responding to change over following a plan

That is, while there is value in the items on the right, we value the items on the left more.



Manifest

2001 workshop in Snowbird, Utah, USA

Manifesto for Agile Software Development

We are uncovering better ways of developing software by doing it and helping others do it.
Through this work we have come to value:

- Individuals and interactions over processes and tools
- Working software over comprehensive documentation
- Customer collaboration over contract negotiation
- Responding to change over following a plan

That is, while there is value in the items on the right, we value the items on the left more.

1. Najważniejsza satysfakcja klienta
2. Witamy zmieniające się wymagania 😊
3. Dostarczamy przyrostowo działające oprogramowanie
4. Usuwamy bariery między pracownikami i odpowiednio motywujemy
5. Samoorganizujące się zespoły

Dlaczego AGILE?

Metodologie sterowane planem

- ✓ Praktyki inżynierskie (mechanika, inż. lądowa)
- ✓ Zaplanuj zanim zbudujesz
- ✓ Szczegółowy plan konstrukcji/budowy
- ✓ Ustalenie harmonogramu i budżetu dla budowy
- ✓ Etap budowy raczej manualny niż intelektualny
- ✓ Szczegółowy harmonogram budowy dla ludzi z mniejszymi umiejętnościami
- ✓ Należy rozdzielić projektowanie od wytwarzania/budowy
- ✓ Płynne przejście od projektu do budowy (UML?)



Martin Fowler

The New Methodology

Dlaczego AGILE?

Budowa oprogramowania

- ✓ Jak trudno zbudować projekt w UMLu gotowy do przekazania programistom?
- ✓ Jaki jest koszt projektowania?
 - Budowa mostu: 10% koszt projektu
 - Software: 15% kosztów to kodowanie + testowanie unitów; 50% koszt projektu (bez testowania) !!!
- ✓ Budowa oprogramowania – zautomatyzowana (kompilator), darmowa



Martin Fowler

The New Methodology

Dlaczego AGILE?

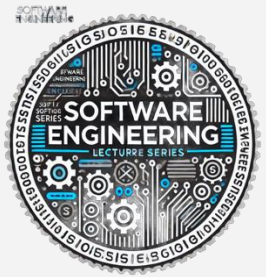
Budowa oprogramowania

- ✓ Cały wysiłek to projektowanie i dlatego wymaga kreatywnych i utalentowanych ludzi
- ✓ Procesy kreatywne nie dają się zaplanować, a więc nie mogą być przewidywalne
- ✓ Metody tradycyjnej inżynierii zawodzą



Martin Fowler

The New Methodology



Nieprzewidywalność wymagań



Jak ocenić koszt realizacji wymagania?



Projektowanie trudno zaplanować i wycenić



Zależy od ludzi realizujących zadanie

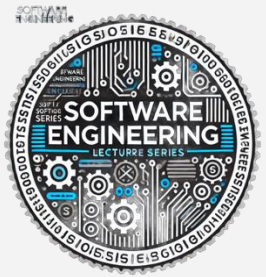


Być logiczny (jak ocenić wartość cechy przed zobaczeniem jej w akcji?)



Wartość podlega szybkim zmianom rynkowym





Nieprzewidywalność wymagań



Dziwne jeśli ktoś się dziwi, że wymagania się zmieniają



Zmiany to norma, a problem: jak sobie z nimi radzić?



Cały proces tworzenia oprogramowania bazuje na wymaganiach



Brak niezmiennych wymagań $\Rightarrow$ brak przewidywalnego planu



Kiedy wymagania da się przewidzieć?

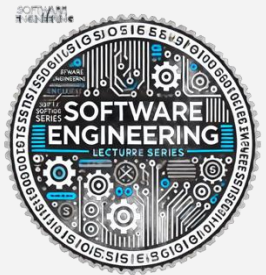


Da się np. w NASA: pełno ceregieli, mnóstwo czasu, duży zespół



Generalnie: NIE!





Nie bój się!



Przewidywalność jest bardzo pożądana ...



... ale nie lódź się, że możesz przewidzieć coś, czego się nie da



Najtrudniejsze jest przyznanie, że istnieje problem



Rezygnacja z przewidywalności nie oznacza powrotu do chaosu!



Potrzebny proces, który pozwoli kontrolować nieprzewidywalność



Kontrola zmian?

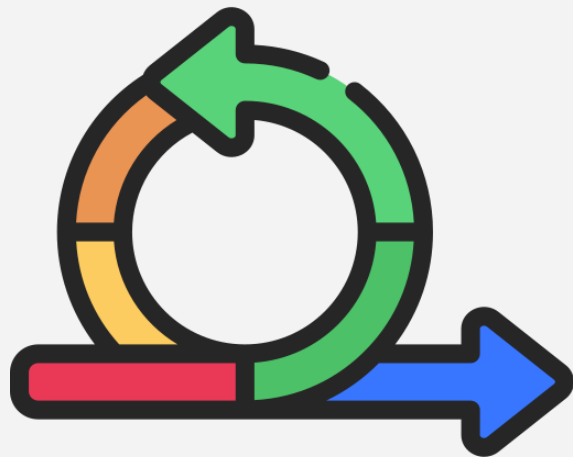
ADAPTACJA



Kontrola zmian?

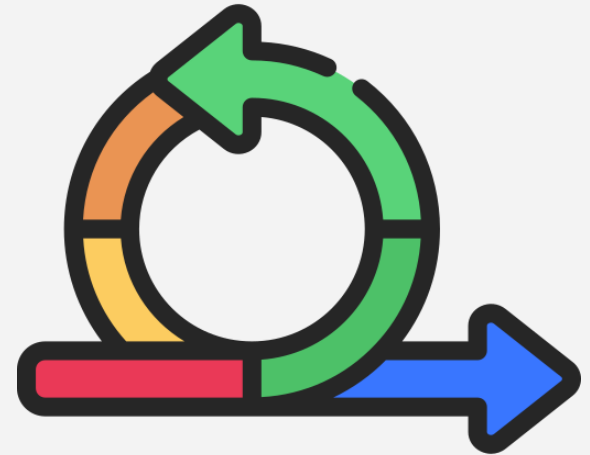
ITERACJE

- ✓ Odpowiedź na to, gdzie jesteśmy
- ✓ Krótkoterminowe planowanie
- ✓ Jak długa powinna być iteracja?
 - XP: 1-2 tygodnie
 - SCRUM: 1 miesiąc
 - ...
 - Tak krótka, jak się da 😊



Kontrola zmian?

- ✓ Ustalona cena wymaga ustalonych wymagań, a więc przewidywalnego procesu
- ✓ Adaptujący się klient
- ✓ Można ustalić budżet
- ✓ Nie można ustalić czasu, ceny i zakresu projektu
- ✓ Podejście **ZWINNE**
- ✓ Ustalenie czasu i ceny
- ✓ Zakres zmienia się w sposób kontrolowany



Klient

- ✓ Aktywnie uczestniczy w procesie tworzenia oprogramowania
- ✓ Otrzymuje produkt lepiej spełniający jego wymagania
- ✓ Lepsza kontrola ryzyka



Ocena projektu

- ✓ Ocena zaplanowanego z góry projektu

Jak mieści się w planie?

Na czas i w kosztach – SUKCES

Brak motywacji

- ✓ Podejście **ZWINNE**

Istotna jest wartość biznesowa

Czy klient otrzymał produkt dla niego bardziej wartościowy niż poniesione koszty?



Czynnik ludzki

- ✓ Ludzie plug-and-play

Nie są istotne osoby, tylko role (analityk, programista, tester itd.)

Czy ludzie są wymiennymi komponentami?

Brak motywacji

- ✓ Podejście **ZWINNE**

Ludzie to niewymienne zasoby

Ludzie to jednostki, od których zależy sukces projektu



Odpowiedzialni i profesjonalni

- ✓ Robotnicy w fabryce z XIX wieku

Nie są w stanie ocenić jak najlepiej wykonać swoją pracę

Niezbyt inteligentni i kreatywni

- ✓ Tworzenie oprogramowania

Bystrzy i zdolni, sława i pieniądze

Trzeba dbać o specjalistę

Odpowiedzialni profesjonaliści decydujący jak wykonać zadania techniczne



Proces zorientowany na ludzi

- ✓ Wymaga poświęcenia i zaangażowania jednostek

- ✓ Deweloperzy

 - Dzielą odpowiedzialności z kierownictwem

 - Podejmują samodzielnie wszystkie techniczne decyzje

 - Oceniają czas realizacji zadań

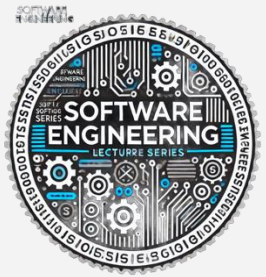
 - Mają dostęp do eksperta

- ✓ Szybka dewaluacja technicznej wiedzy

 - Dzielenie pracy i zaufanie w zespole

- ✓ Samoadaptujący się proces

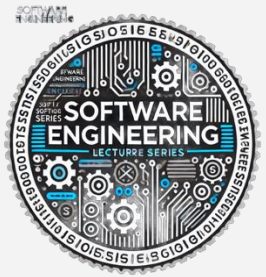




Dla kogo podejście zwinne?

-  Zespół musi chcieć tak pracować
-  Klient musi chcieć tak pracować
-  Łatwiej zacząć od prostszego procesu
-  Zaczniij od prostszego projektu
-  Dobry mentor wart swojej wagi w złocie
-  Słuchaj mentora





Kiedy złe jest podejście zwinne?



Zależy od nastawienia ludzi



Czy są jakieś granice stosowania?





Manifest 2001 workshop in Snowbird, Utah, USA

Manifesto for Agile Software Development

We are uncovering better ways of developing software by doing it and helping others do it.
Through this work we have come to value:

Individuals and interactions over processes and tools

Working software over comprehensive documentation

Customer collaboration over contract negotiation

Responding to change over following a plan

That is, while there is value in the items on the right, we value the items on the left more.



Manifest krytycznym okiem

Manifesto for Agile Software Development

We are uncovering better ways of developing software by doing it and helping others do it.
Through this work we have come to value:

Individuals and interactions over processes and tools
Working software over comprehensive documentation
Customer collaboration over contract negotiation
Responding to change over following a plan

That is, while there is value in the items on the right, we value the items on the left more.

1. Rozmawianie z ludźmi pozwala nam robić to, co chcemy, w taki sposób jak chcemy. Oczywiście zakładamy, że wiemy czego chcesz, nawet jeśli ty tego nie wiesz.
2. Cały czas chcemy kodować. Prawdziwi programiści nie piszą dokumentacji.

Rakitin, S., Manifesto Elicits Cynicism. IEEE Computer, 2001. 34(12)



Manifest krytycznym okiem

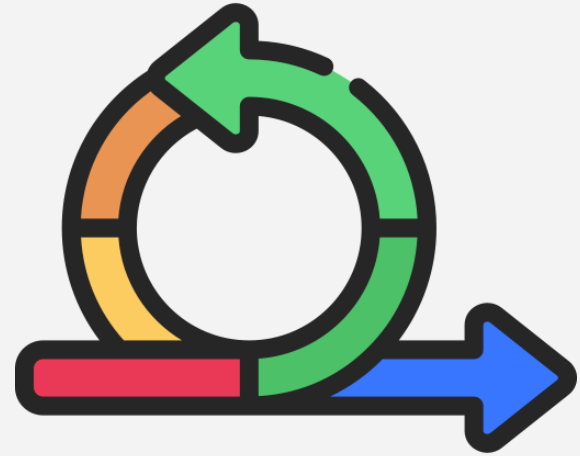


3. Nie marnujemy czasu na targowaniu się o szczegóły, to tylko uniemożliwia spędzenie całego czasu na kodowaniu. Rozwikłamy wątpliwości, jak coś dostarczymy..
4. Przestrzeganie planu wymagałoby od nas przemyślenia problemu i sposobów jego rozwiązania. Po co mielibyśmy to robić, skoro możemy kodować?

Rakitin, S., Manifesto Elicits Cynicism. IEEE Computer, 2001. 34(12)

Metodyki ZWINNE

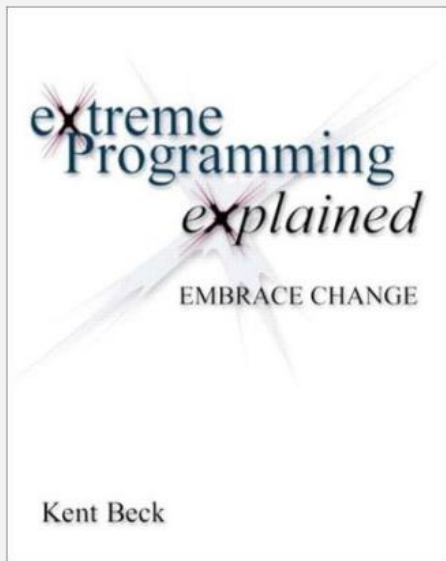
- 01** eXtreme Programming (XP)
- 02** SCRUM
- 03** Crystal
- 04** Lean Development
- 05** Adaptive Software Development (ASD)
- 06** Dynamic Systems Development Method (DSDM)/ Framework for Business Agility
- 07** Feature Driven Development (FDD)
- 08** Kanban / SAFe /LeSS



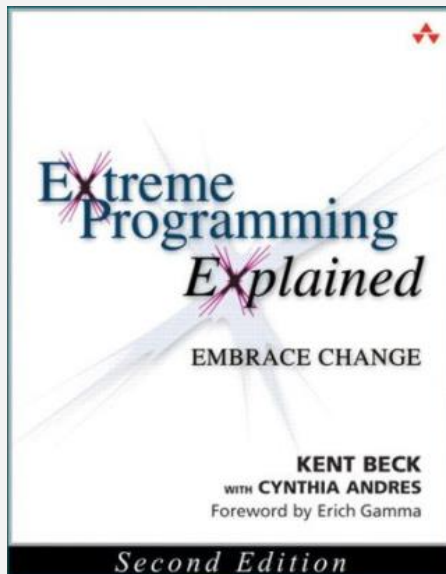


06.01

eXtreme Programming (XP)



1999



2004



Kent Beck

Chrysler C3 project, 1997

4 wartości

01 Prostota

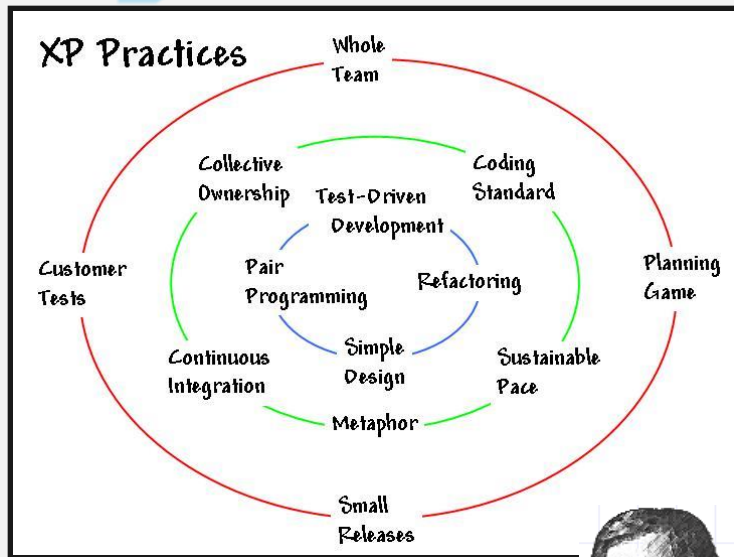
02 Komunikacja

03 Informacja zwrotna

04 Gotowość do zmian



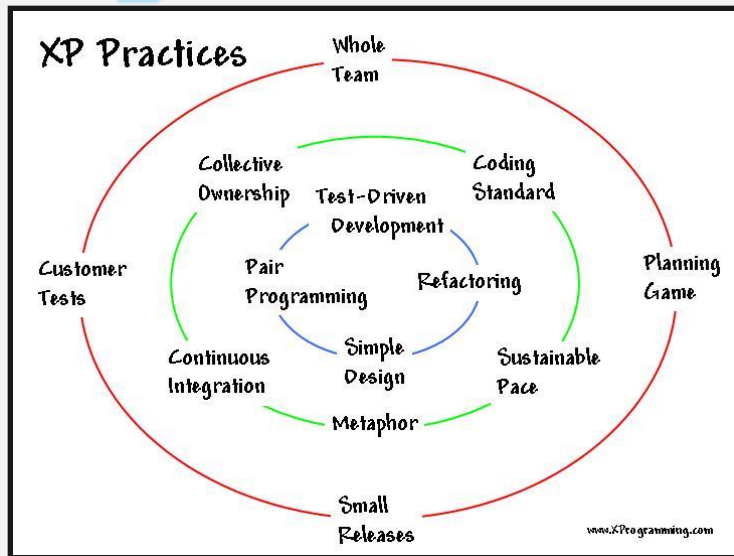
Praktyki



Ron Jeffries



Praktyki : KLIENT



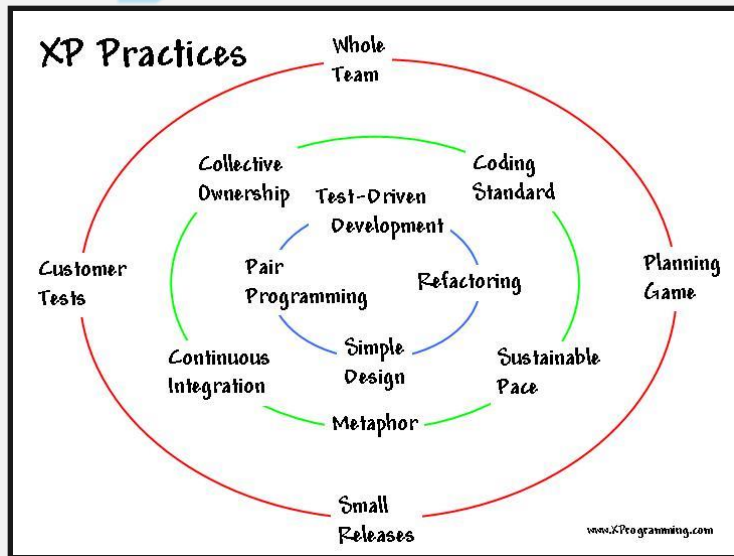
Whole Team:

- > Każdy uczestniczący w projekcie jest członkiem zespołu
- > Zespół tworzony wokół Klienta

Planning Game, Small Releases, Customer Tests:

- > Proste planowanie i śledzenie projektu
- > Wartość biznesowa przede wszystkim
- > Małe w pełni zintegrowane wdrażanie oprogramowania, które przeszło testy Klienta

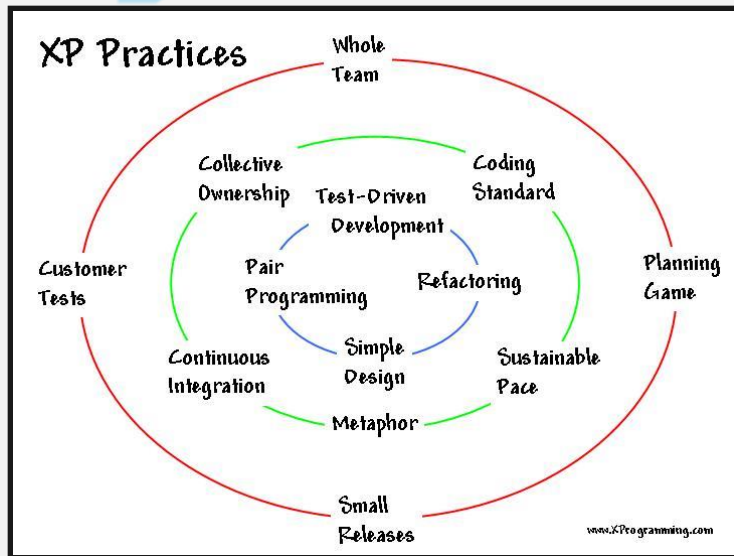
Praktyki : ZESPÓŁ



Simple Design, Pair Programming, Test-Driven Development, Refactoring:

- >Praca grupowa i w parach
- >Prosty projekt
- >Obsesyjne testowanie
- >Ciągłe udoskonalanie projektu by spełniał bieżące wymagania

Praktyki : KODOWANIE



Continuous Integration, Collective Code Ownership, Coding Standard:

- > Stale działający i zintegrowany system
- > Współdzielenie kodu (praca w parach)
- > Ustandaryzowane kodowanie

Metaphor, Sustainable Pace:

- > Wspólny prosty obraz systemu
- > Każdy pracuje ze stałą nigdy nie słabnącą wydajnością (szybkością)



www.extremeprogramming.org



The Rules and Practices of Extreme Programming.

*Lessons
Learned*

Planning

- ❖ ❖ User stories are written.
- ❖ ❖ Release planning creates the schedule.
- ❖ ❖ Make frequent small releases.
- ❖ ❖ The Project Velocity is measured.
- ❖ ❖ The project is divided into iterations.
- ❖ ❖ Iteration planning starts each iteration.
- ❖ ❖ Move people around.
- ❖ ❖ A stand-up meeting starts each day.
- ❖ ❖ Fix XP when it breaks.

Designing

- ❖ ❖ Simplicity.
- ❖ ❖ Choose a system metaphor.
- ❖ ❖ Use CRC cards for design sessions.
- ❖ ❖ Create spike solutions to reduce risk.
- ❖ ❖ No functionality is added early.
- ❖ ❖ Refactor whenever and wherever possible.

Coding

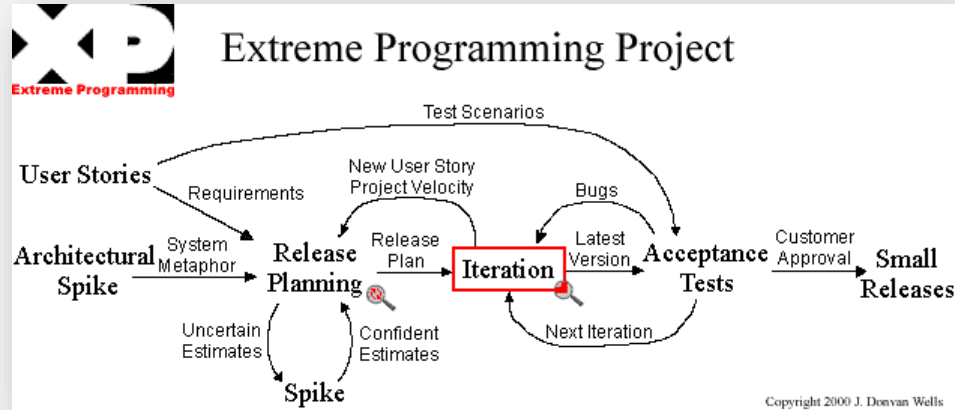
- ❖ ❖ The customer is always available.
- ❖ ❖ Code must be written to agreed standards.
- ❖ ❖ Code the unit test first.
- ❖ ❖ All production code is pair programmed.
- ❖ ❖ Only one pair integrates code at a time.
- ❖ ❖ Integrate often.
- ❖ ❖ Use collective code ownership.
- ❖ ❖ Leave optimization till last.
- ❖ ❖ No overtime.

Testing

- ❖ ❖ All code must have unit tests.
- ❖ ❖ All code must pass all unit tests before it can be released.
- ❖ ❖ When a bug is found tests are created.
- ❖ ❖ Acceptance tests are run often and the score is published.

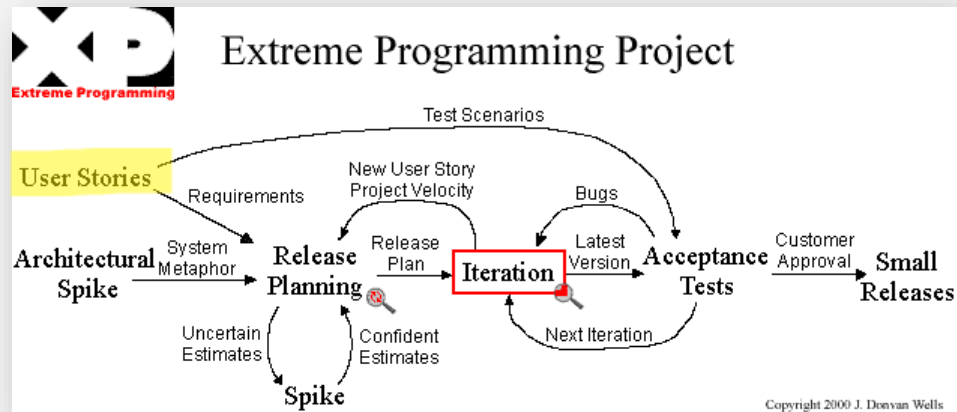


www.extremeprogramming.org





www.extremeprogramming.org



Zwinna specyfikacja wymagań

01 Karty wymagań (User Stories)

02 Szkice ekranów

03 Testy akceptacyjne



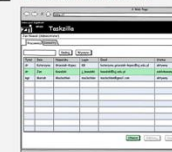
Zwinna specyfikacja wymagań

01 Karty wymagań (User Stories)

Jako [osoba odgrywająca daną rolę] **chciałbym móc** [wykonać jakąś czynność] **aby** [osiągnąć jakiś cel].

Zwinna specyfikacja

02 Szkice ekranów



Zwinna specyfikacja wymagań

03 Testy akceptacyjne

UTWÓRZ KONTO testy akceptacyjne

- Administrator wypełnił prawidłowo wszystkie wymagane pola, wprowadził również unikalną wartość identyfikatora użytkownika – w nowo otwartym oknie system informuje o pomyślnym utworzeniu konta i wysłaniu wiadomości e-mail
- Administrator wypełnił wszystkie wymagane pola, jednak w systemie figuruje już użytkownik o takim samym identyfikatorze – system wyświetla na formularzu komunikat ostrzegawczy o tym, że pole „login” musi zawierać unikalną wartość.
- Administrator nie wypełnił wymaganych pól (oznaczonych *) – system wyświetla na formularzu informację o niekompletnych danych wraz z prośbą o ich uzupełnienie.



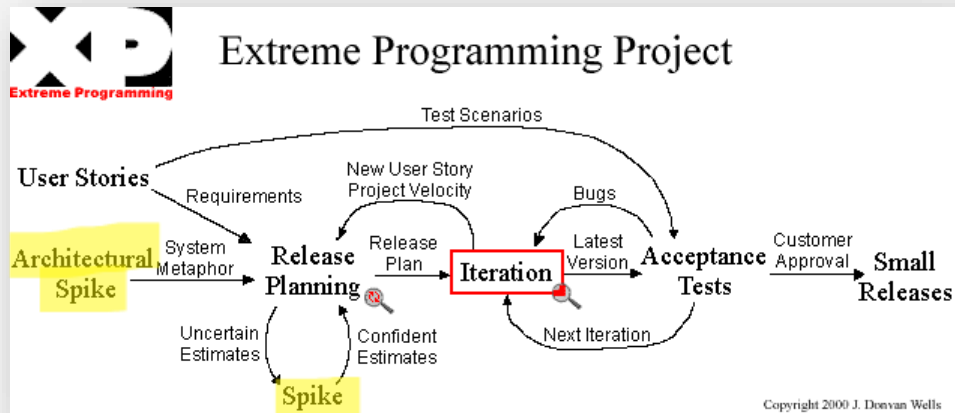
>Każdy scenariusz jest szacowany na 1, 2 albo 3 tygodnie „idealnego programowania”

> Mniej niż tydzień to zbyt niski poziom szczegółowości

> 80 ± 20 US to idealna liczba by stworzyć Release Plan

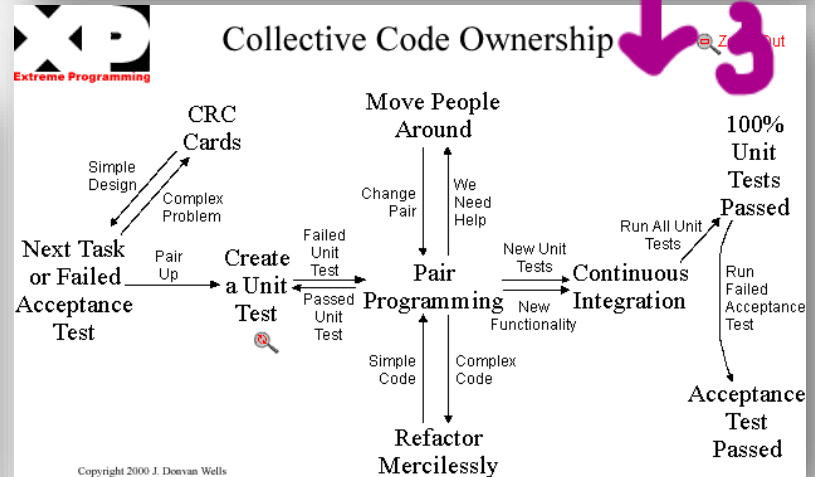
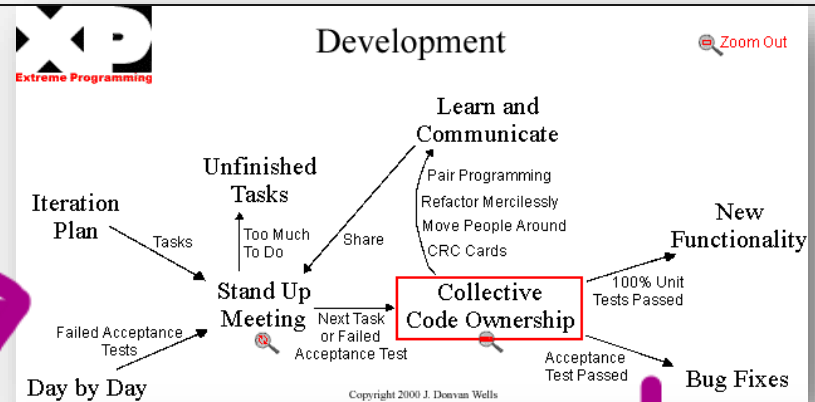
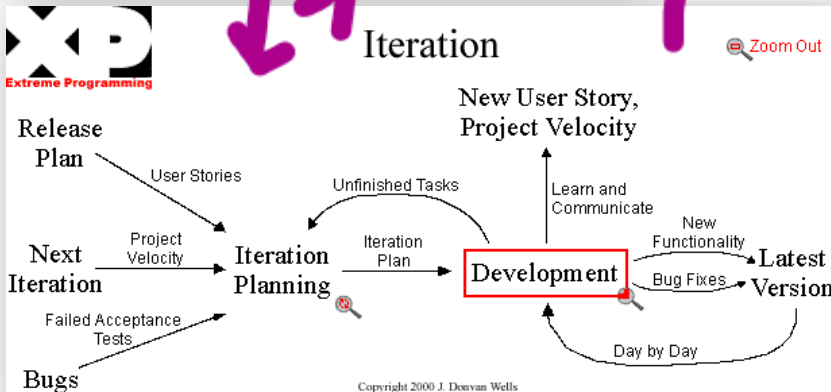
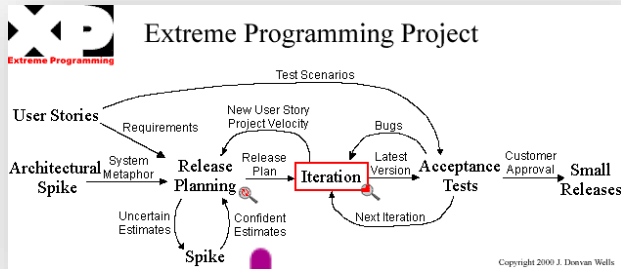


www.extremeprogramming.org



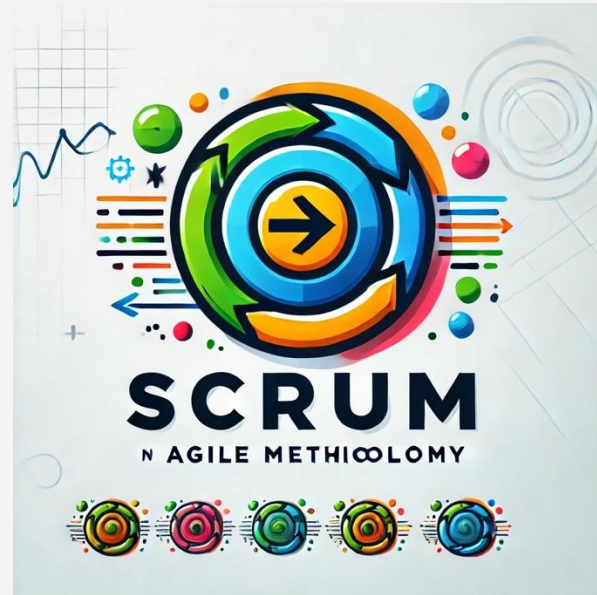
Spike Solution:

- > Tworzone by rozwiązać problemy techniczne bądź projektowe
- > Jest to bardzo prosty program do zbadania potencjalnych rozwiązań
- > Throw-away prototyping
- > Tworzone by:
 - > zredukować ryzyko rozwiązań technicznych
 - > zwiększyć wiarygodność oszacowania US



06.02

SCRUM



Polska grupa SCRUM

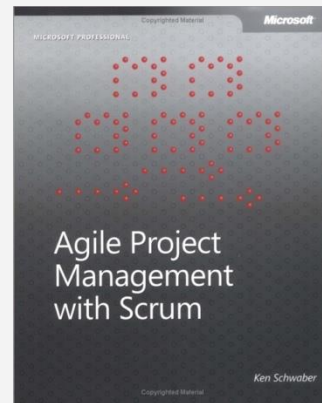
<http://scrum.org.pl>



Ken Schwaber

Zaprezentowany OMG w **1995** przez Advanced Development Methods (ADM) and VMARK Software (VMARK)

Niewydajne prowadzenie projektów
zorientowanych obiektowo



SCRUM

Nazwa formacji w rugby – MŁYN

AGRESYWNE usuwanie przeciwności!

Zespołowa praca w zwartym szyku,
nastawiona na realizację celu

Ciągła i dynamiczna adaptacja



Metodyka zarządzania projektem często stosowana z XP

SCRUM

Scrum jest wzorcem!!!

Małe zespoły (< 10 osób)

Serie sprintów

Rozwój inkrementacyjny

Time-boxing

Koncentracja na zarządzaniu projektem

Mały nacisk na praktyki

Łączenie z praktykami XP



SCRUM

scrum.org.pl

ZESPÓŁ SCRUM

PRODUCT OWNER



DEVELOPERZY



SCRUM MASTER



PRODUCT OWNER



Product Owner
odpowiada za wartość
produktu i decyduje o tym
jakie zmiany w nim
zostaną wykonane i w
jakiej kolejności.

SCRUM

[scrum.org.pl](https://www.scrum.org.pl)

ZESPÓŁ SCRUM

PRODUCT OWNER



DEVELOPERZY



SCRUM MASTER



DEVELOPERZY



Developerzy odpowiedzialni są za wytworzenie i dostarczenie co iterację (Sprint) kompletnego, działającego, gotowego produktu i posiadają wspólnie wszystkie kompetencje niezbędne, aby taki produkt wytworzyć.

SCRUM

[scrum.org.pl](https://www.scrum.org.pl)

ZESPÓŁ SCRUM

PRODUCT OWNER



DEVELOPERZY



SCRUM MASTER



SCRUM MASTER



Scrum Master odpowiada za efektywność działania Zespołu Scrum i za to, by Scrum był prawidłowo rozumiany i stosowany zarówno w samym Zespole jak i w organizacji.

SCRUM

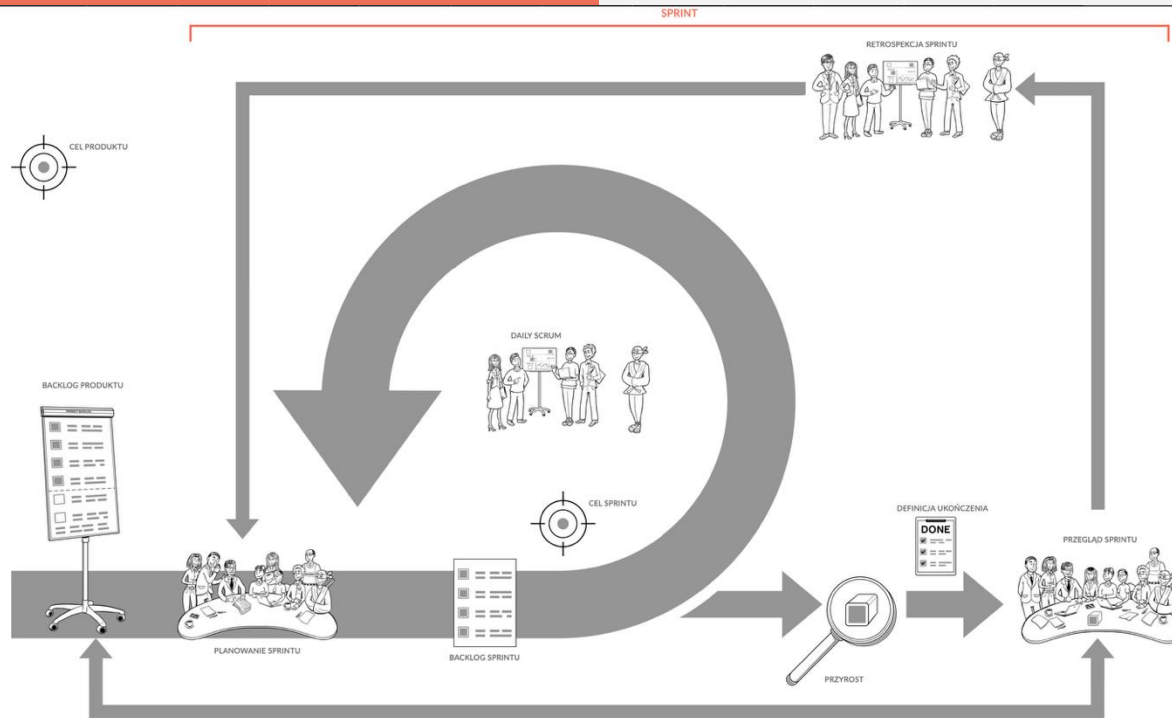
[scrum.org.pl](https://www.scrum.org.pl)

Proces:

Zbiór luźno powiązanych czynności łączących znane i działające techniki oraz narzędzia, pozwalający zbudować system

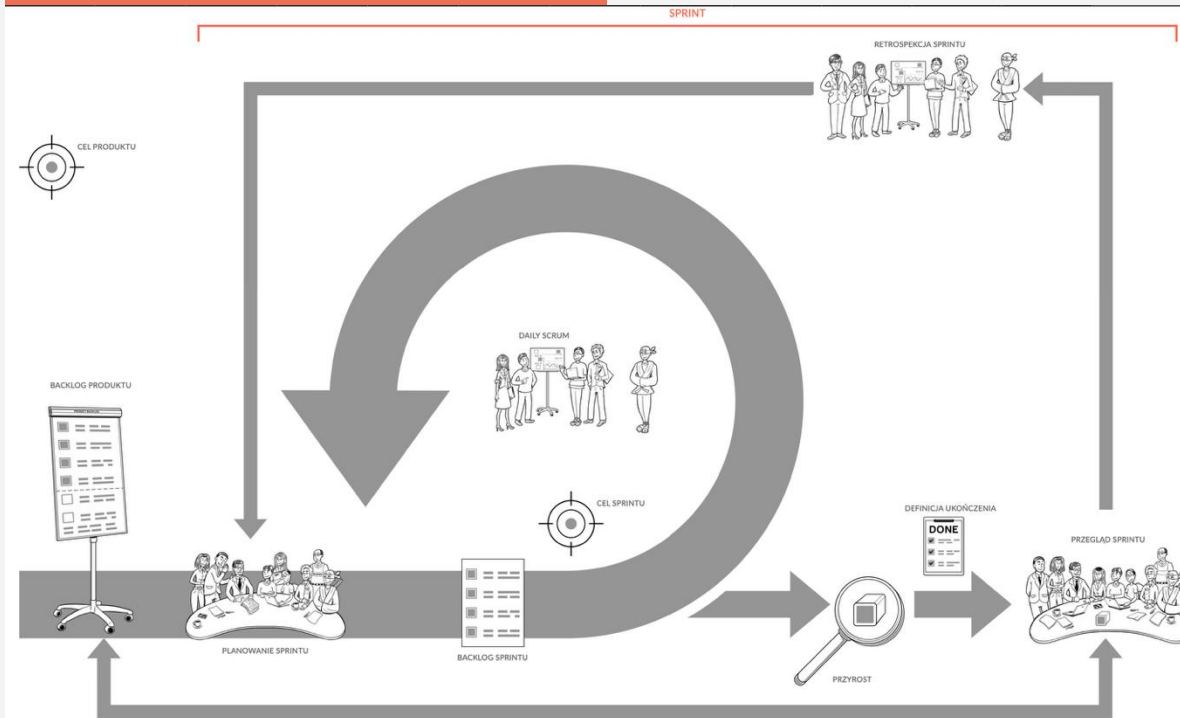
Dwie fazy:

1. Planowanie
2. Sprint



SCRUM

scrum.org.pl



Sprint:

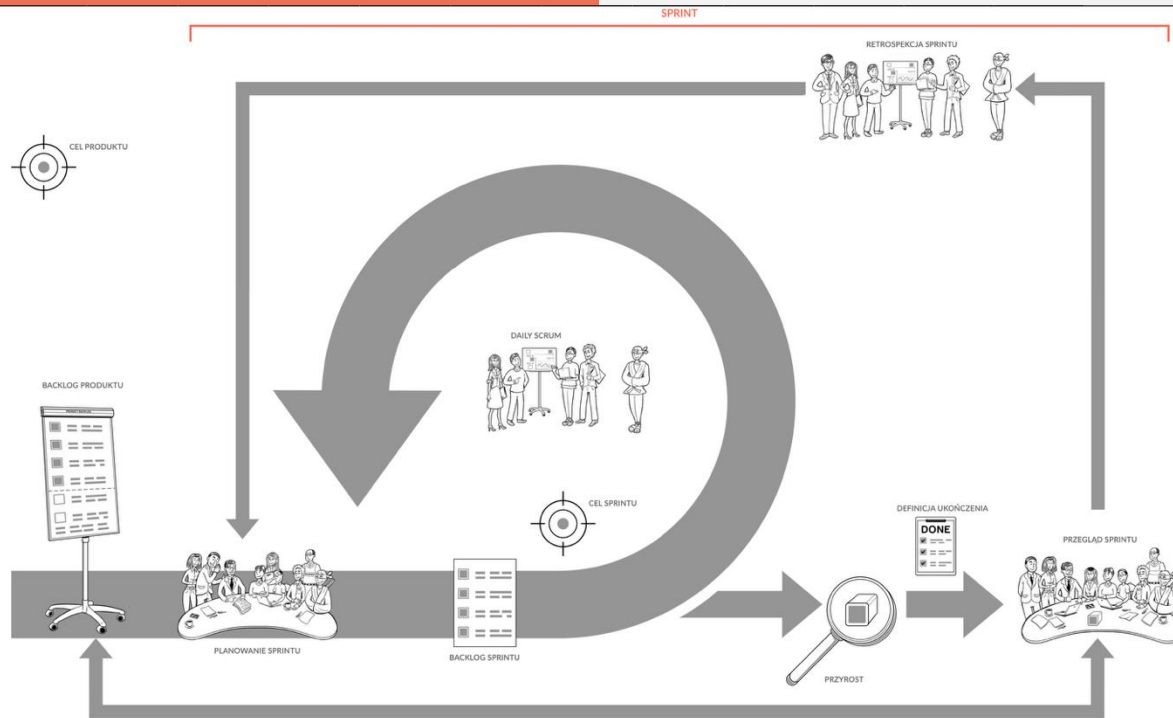
Iteracja 2-4 tygodnie

Zadania przypisane do członków zespołu

Koncentracja na celu:
> żadnego rozpraszania uwagi
> zero zmian/przerwań z zewnątrz
> mogą być zidentyfikowane nowe zadania

SCRUM

scrum.org.pl



Sprint:

Częste, krótkie spotkania
(Scrum Meetings)

Każdy zespół dostarcza
widocznego, działającego
elementu

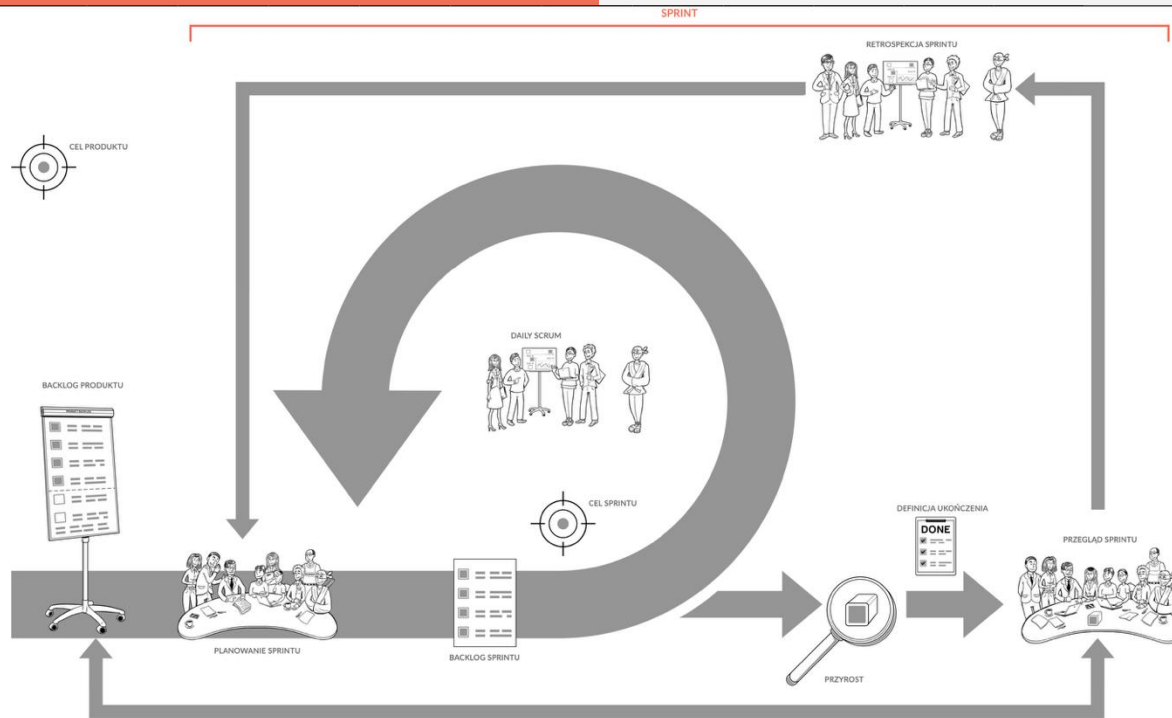
Kolejne iteracje są
inkrementacyjne

Jasno zdefiniowane
obowiązki i wyniki iteracji

Cały zespół jest
zaangażowany

SCRUM

scrum.org.pl



SCRUM meeting:

Codzienne spotkania 15–30 minut

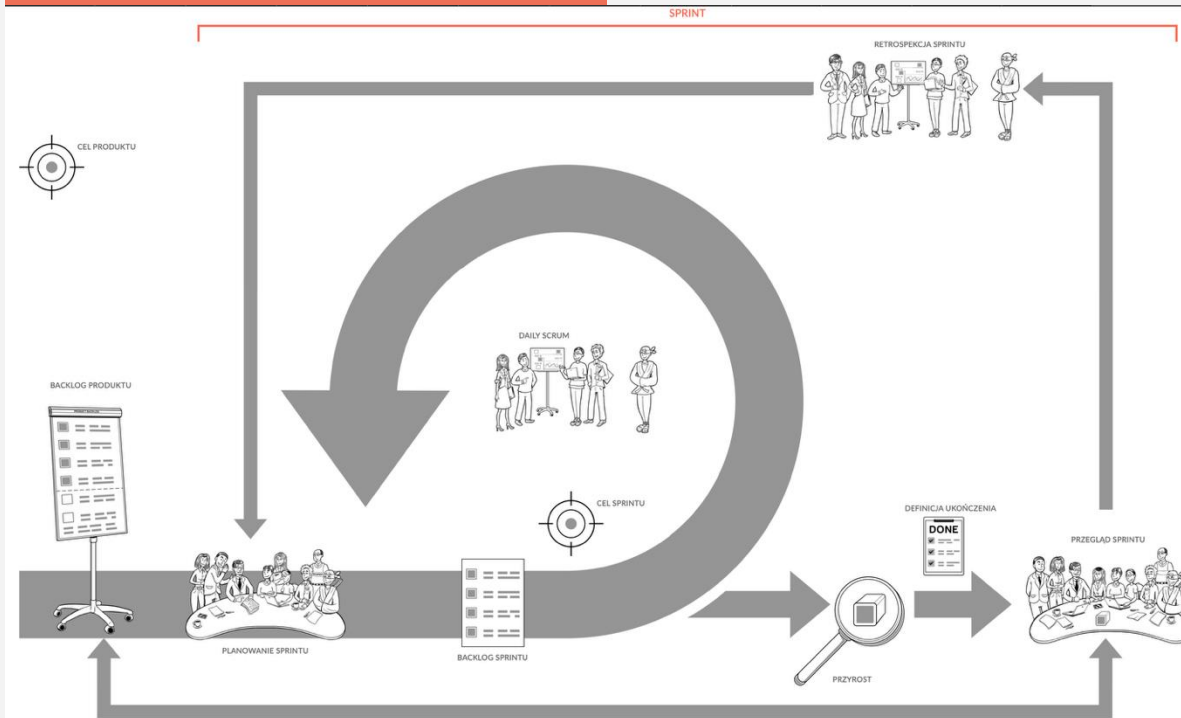
Cały zespół bierze udział

Do każdego zadania 3 pytania:

1. Co zrobiłeś od ostatniego spotkania?
2. Jakie napotkałeś trudności?
3. Co zrobisz do następnego spotkania?

SCRUM

[scrum.org.pl](https://www.scrum.org.pl)



Koniec sprintu:

Przegląd/spotkanie co się
działo

Planowanie kolejnego
sprintu

Dostarczenie działającej
części produktu

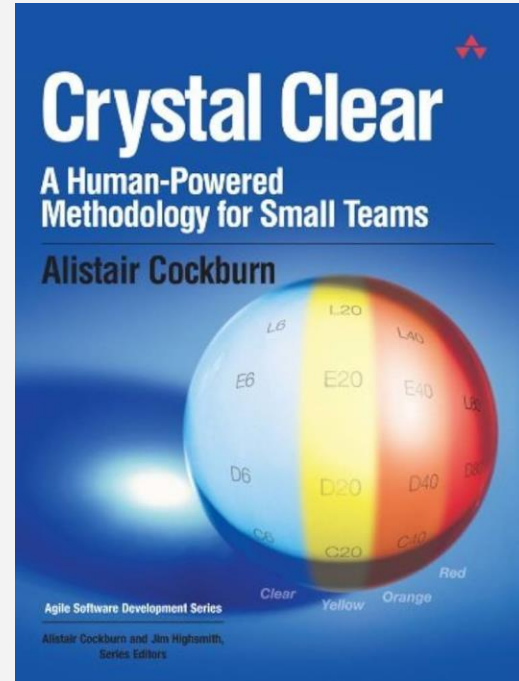
Zebranie niespodzianek

WSZYSTKO może zostać
zmienione (zakres prac,
priorytety)

Projekt może zostać
przerwany

06.03

Crystal Family

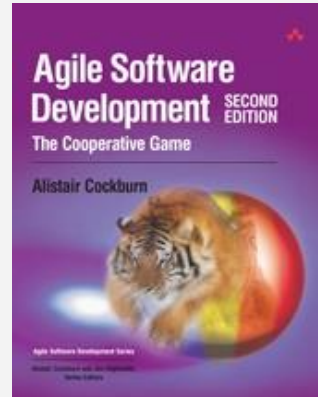




Alistair Cockburn

alistair.cockburn.us

- > Opracowanie metodologii dla the IBM Consulting Group
- > 1991 – 1999, Hong Kong – obie Ameryki,
- > Norwegia i Ameryka Południowa
COBOL, Smalltalk, Java, VB, Sapiens, Synon





Alistair Cockburn

alistair.cockburn.us

01. Rozmowy twarzą w twarz zamiast dokumentów

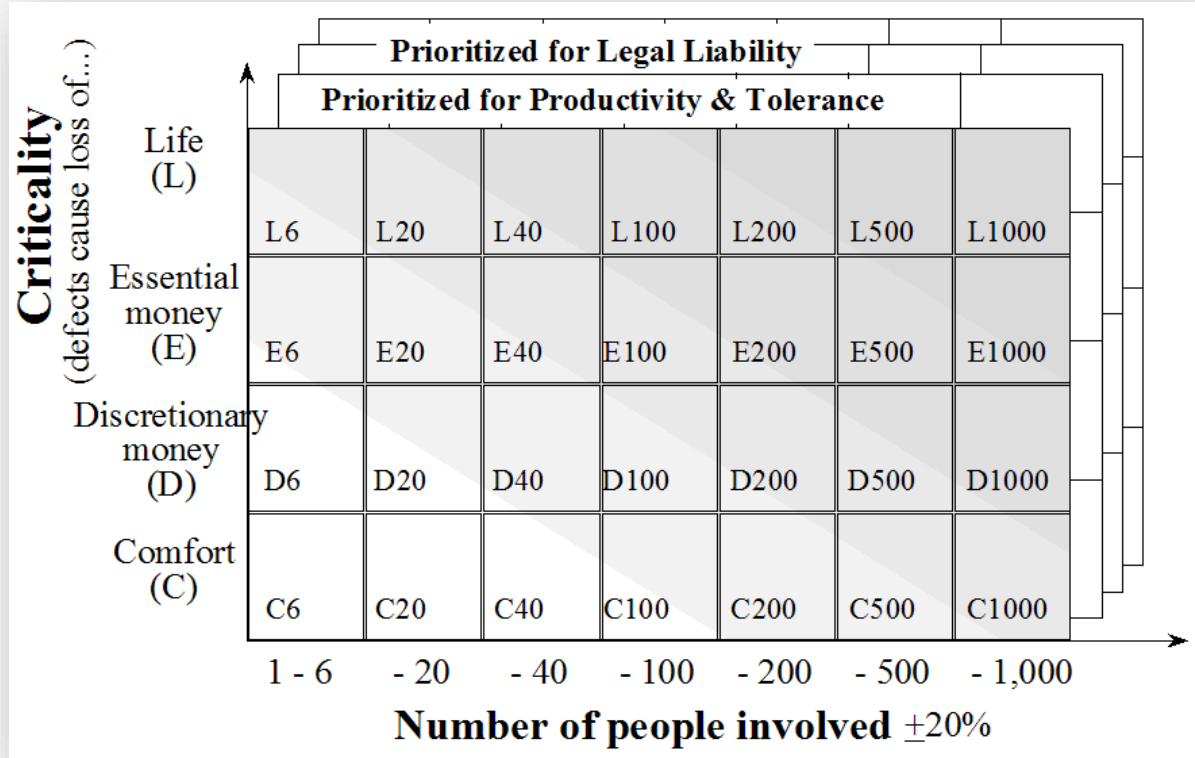
02. Im częściej dostarczane są małe, działające części systemu, tym pewniejsza pomyślna realizacja systemu

03. Różne projekty mają różne potrzeby

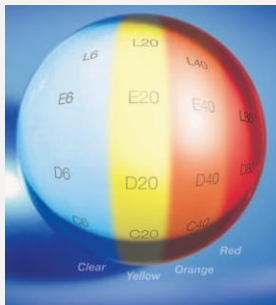


Alistair Cockburn
alistair.cockburn.us

Wniosk



The Crystal Light Family



E6	E20	E40	E80
D6	D20	D40	D80
C6	C20	C40	C80



Crystal Clear: 2-6 osób



Crystal Yellow: 6-20 osób



Crystal Orange: 20-40 osób



Crystal Red: 40-100 osób



Crystal Magenta, Crystal Blue, etc.

Przesunięcie wzdłuż osi Y to przejście z wersji **light** w kierunku **hard**

Niespodzianki



Jak mało potrzeba procesu i kontroli by zespół dobrze funkcjonował



Krytyczne dla zarządzania projektem jest zrozumienie i informowanie o
współzależnościach

Często nie potrzebne są narzędzia
Zaufanie i komunikacja!!!!!!!



Crystal Light wygląda ciężko przy XP





Alistair Cockburn

alistair.cockburn.us

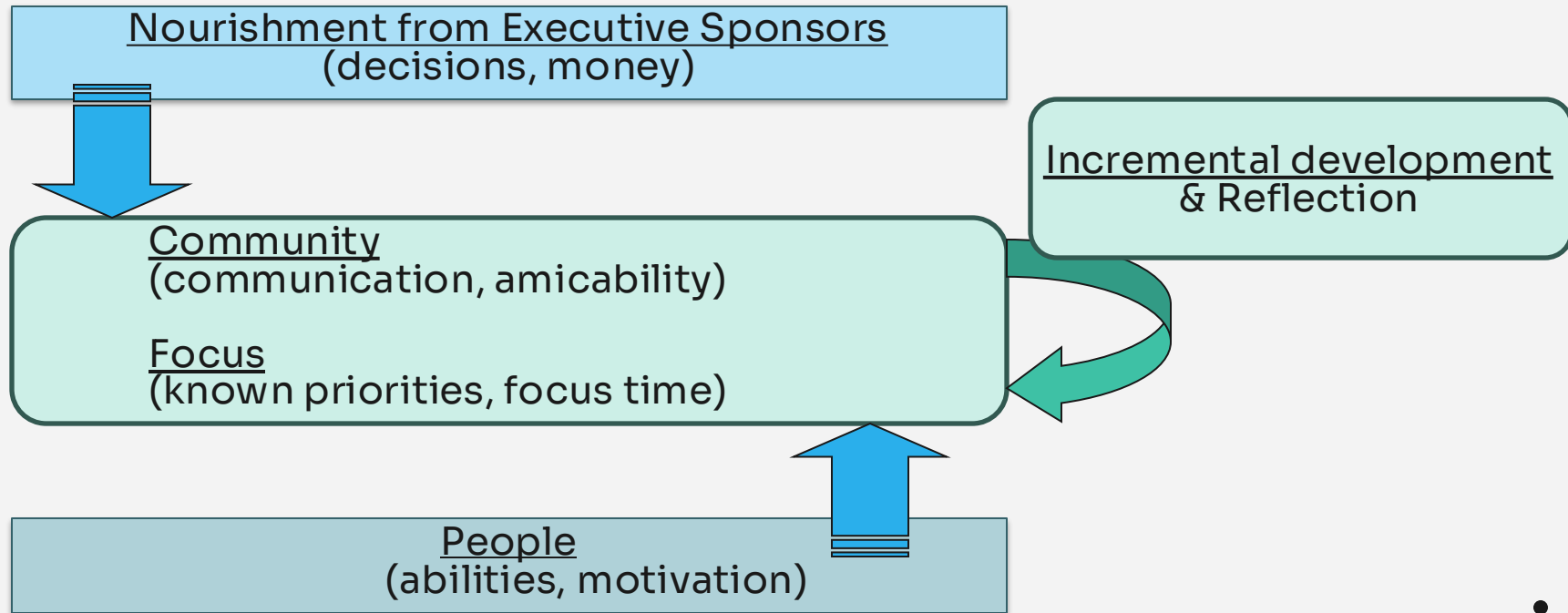
Software development is a (series of) **cooperative game(s)**... The endpoint of the game is an operating software system... The next game is the alteration or replacement of the system, or creation of a neighboring system .

10 Krytycznych czynników sukcesu

28.11

7 własności projektów zakończonych sukcesem

10 czynników sukcesu



7 praktyk sukcesu



- 01** Częste iteracje (time-boxing; 2x0.5 roku)
- 02** Przemyślane udoskonalanie: krytykuj i naprawiaj
- 03** Osmotyczna komunikacja i zdobywanie wiedzy: odpowiednia organizacja i kanały komunikacji
- 04** Osobiste bezpieczeństwo: można być szczerym i się nie zgadzać
- 05** Koncentracja: jasne zadania i priorytety, bez nadmiernego obciążenia pracą
- 06** Eksperti w zasięgu ręki
- 07** Środowisko zwinne: automatyczne testowanie, zarządzanie konfiguracją, częste integracje

06.04

Lean Development



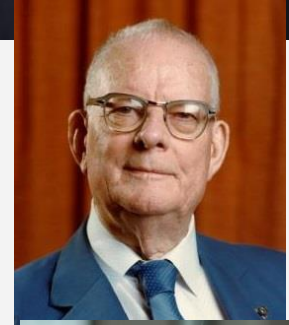
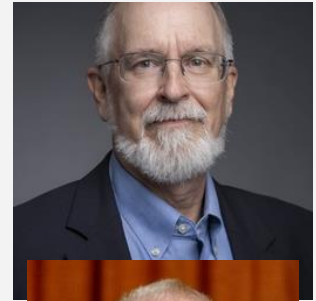
Lean początki

Robert Charette, początek lat 90tych
ITABHI Corporation: Lean manufacturing, Taiichi Ohno,
the Toyota Production System

Just-In-Time
Jidoka

W. Edwards Deming, Total Quality Management (TQM),
lata 80te

Mary Poppendieck, <http://www.poppendieck.com>



14 punktów TQM

- 01** Stale udoskonalaj system
- 02** Likwiduj bariery pomiędzy departamentami
- 03** ...
- 11** Pozbądź się strachu
- 12** Wyeliminuj normy, numeryczne cele i oceny osiągnięć
- 13** Nie oceniaj biznesu wyłącznie na podstawie koszt
- 14** Minimalizuj całkowite koszty



12 zasad LSD

- 01** Najwyższym priorytetem jest satysfakcja klienta
- 02** Zawsze dostarczaj najlepszą jakość za rozsądną cenę
- 03** Sukces zależy od aktywnego udziału klienta
- 04** Tworzenie oprogramowania to praca zespołowa
- 05** Wszystko jest zmienne
- 06** Ogólne, nie specjalistyczne oprogramowanie
- 07** Buduj z komponentów



12 zasad LSD

- 08** 80% rozwiązania dziś zamiast 100% jutro
- 09** Kluczowy jest minimalizm
- 10** Potrzeby definiują technologię
- 11** Rozwój produktu to rozwój funkcjonalności, nie rozmiaru
- 12** Nie stosuj LSD tam, gdzie można użyć innych metod z większym zyskiem



13-ta niepisana zasada LSD



13 75% sukcesu to czynnik ludzki $\Rightarrow$ potrzeba zaangażowanych i zadowolonych pracowników

LSD nie dla każdego



Podejście strategiczne, sterowane zasadami



Większość zwinnych jest taktyczna, zorientowana na zespół

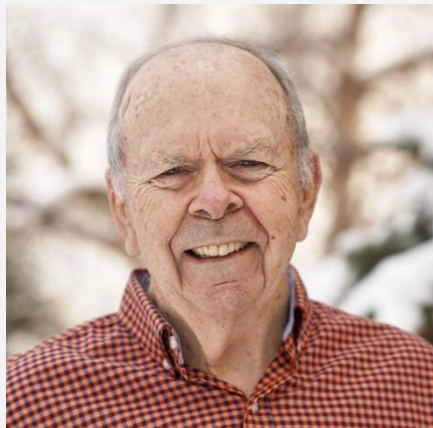


06.05

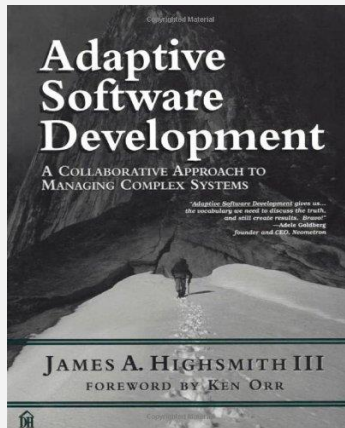


Adaptive Software Development

Nieprzewidywalność wymagań



Jim Highsmith



1999



**Complex Adaptive Systems (CAS)
theory [Holland1995]**

Understanding Adaptive Software Development (ASD): A Comprehensive Overview

CAS Theory



Świat: agenci, środowisko, emergencja

Deweloperzy tworzą środowisko

Deweloperzy są agentami

Oprogramowanie to emergentny
rezultat współpracy i współzawodnictwa
agentów



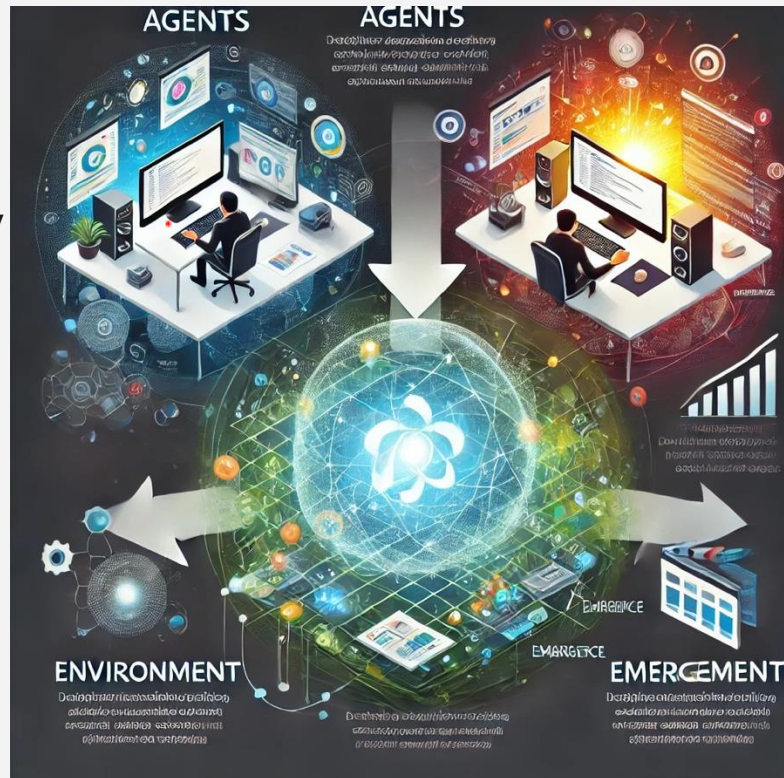
Zarządzanie

ADAPTACJA: szybka odpowiedź na zmiany

The Adaptive Leadership-Collaboration model

Strategia:

- > koncentracja na produkcie, nie na procesie
- > duża skala: dostarczenie narzędzi i technik umożliwiających samoorganizację wirtualnym zespołom



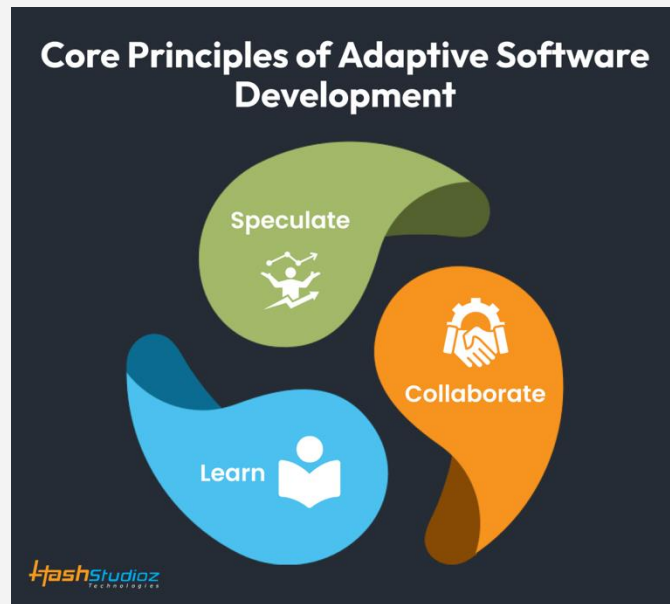
Speculate-Collaborate-Learn

Cykl życia zorientowany na:

zmianę
ciągłe uczenie się
patrzenie w niepewną przyszłość
intensywna współpraca w zespole

Każda iteracja to:

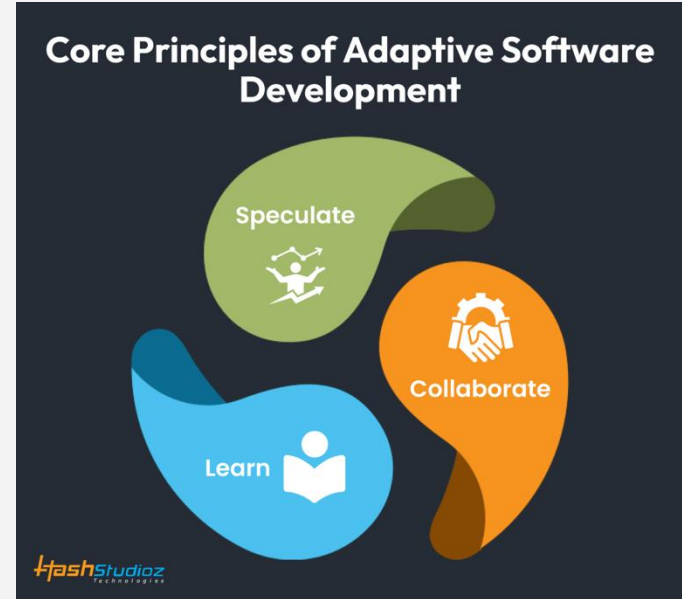
SPEKULACJA
WSPÓŁPRACA
NAUKA



Speculate–Collaborate–Learn

SPEKULACJA:

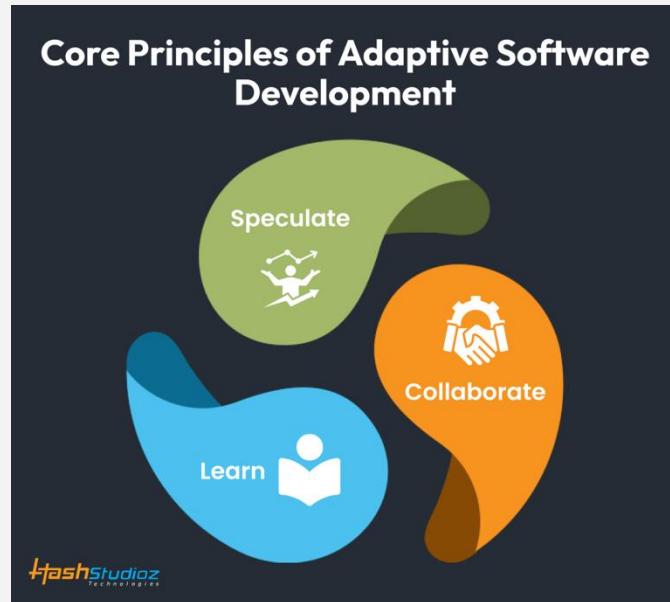
1. Rozpoczęcie projektu – ustalenie misji
2. Ustalenie time-boxu dla całego projektu
3. Ustalenie liczby iteracji i przypisanie im time-boxów
4. Wybranie tematu bądź celu dla każdej iteracji
5. Deweloperzy i klienci przypisują rezultaty każdej iteracji



Speculate-Collaborate-Learn

WSPÓŁPRACA:

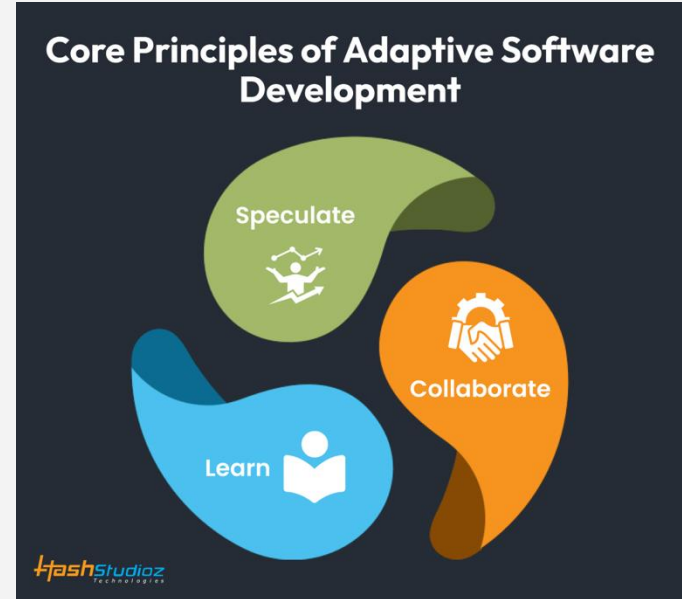
1. Zespół techniczny dostarcza działającego oprogramowania
2. Kierownictwo ułatwia współpracę i współbieżne działania
3. Dobra współpraca:
 - umiejętność pracy w grupie
 - dzielenie wiedzy
 - podejmowanie decyzji



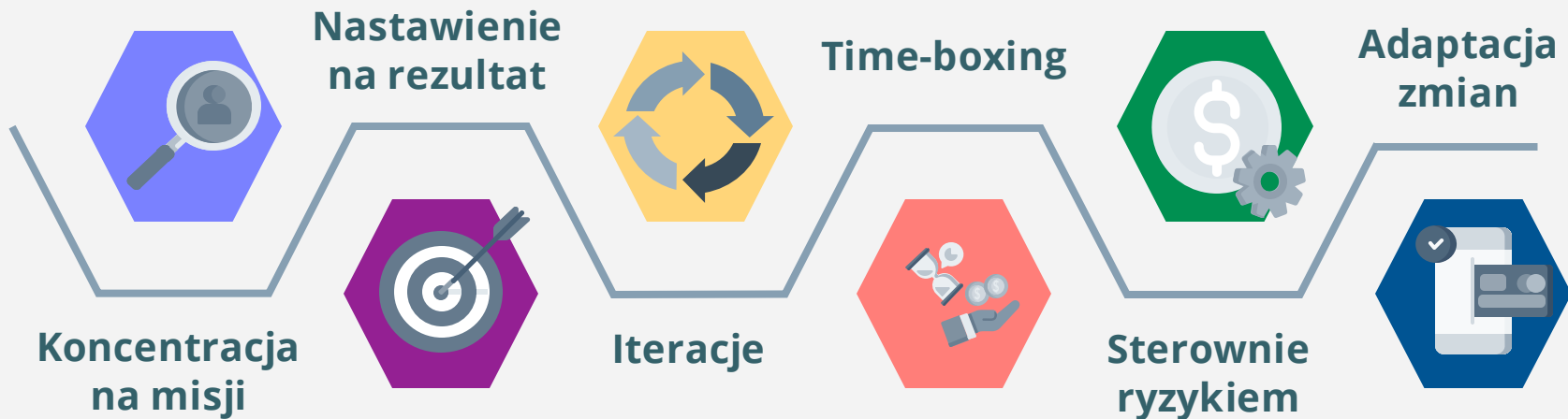
Speculate-Collaborate-Learn

NAUKA:

1. Ocena jakości z punktu widzenia klienta
2. Ocena jakości pod kontem technicznym
3. Ocena funkcjonowania zespołu wdrożeń i praktyk stosowanych przez członków zespołu
4. Ocena statusu projektu



Adaptive Software Development



ASD to strategia nie technika



Żadne działania nie są konieczne



Są wskazówki/oczekiwania Highsmitha



Każda technika ma określony zakres stosowalności



Zawsze można znaleźć sytuację, że nie zadziała?

Understanding Adaptive Software Development (ASD): A Comprehensive Overview



By Manvendra Kunwar 🕒 August 8, 2024



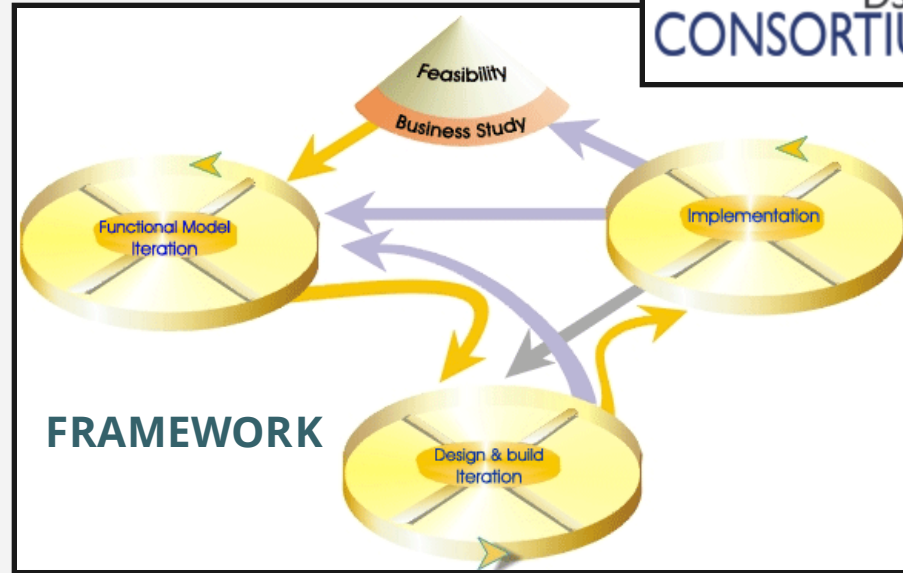
#Adaptive development, #Adaptive Software Development, #Software lifecycle

Adaptive Software Development (ASD) is making significant strides in the industry, as highlighted by recent data:

- 1) **75%** of organizations implementing ASD report a **30% increase** in project delivery speed, showcasing its effectiveness in agile adaptation.
- 2) **60%** of ASD-adopting teams experience a **40% improvement** in collaboration and communication, emphasizing the method's role in fostering teamwork.
- 3) **85%** of ASD projects achieve their goals within budget, compared to **55%** of traditional methods, reflecting enhanced efficiency and cost management.
- 4) **50%** of companies that transitioned to ASD see a **25% reduction** in time-to-market, accelerating their competitive edge.

06.06₍₁₎

Dynamic Systems Development Method



Framework for Business Agility

Agility in an organisation's culture,
leadership and governance

enables

Agility in its capability to operate,
change, and support its business

and optimises

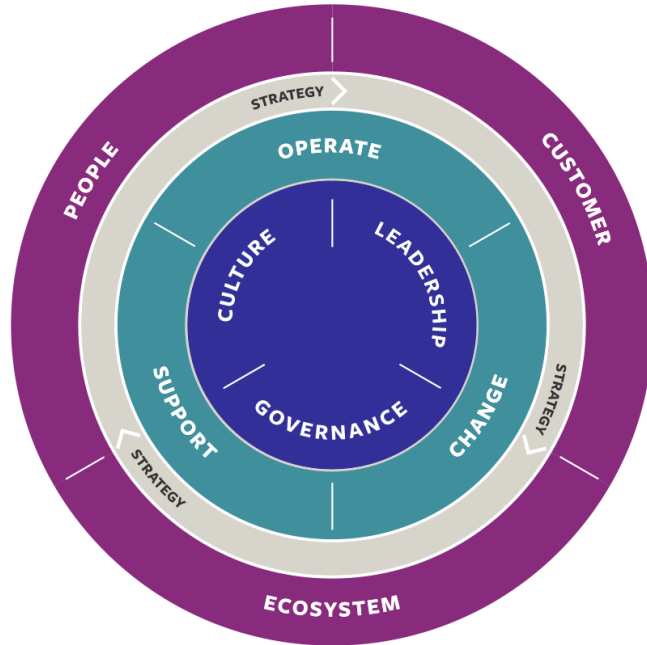
Agility in its delivery of value to its
customers, its ecosystem and its people

Agility in strategy ensures that
the organisation maintains a focus
on delivery of value

06.06₍₂₎

Framework for Business Agility

Agile Business Consortium



Framework for Business Agility

Agility in an organisation's culture,
leadership and governance
enables

Agility in its capability to operate,
change, and support its business
and optimises

Agility in its delivery of value to its
customers, its ecosystem and its people

Agility in strategy ensures that
the organisation maintains a focus
on delivery of value

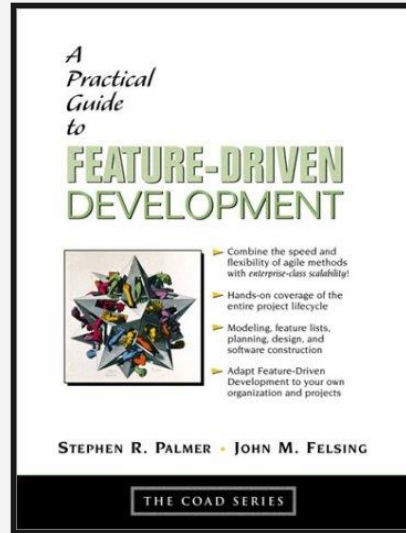
06.07

Feature Driven Development





Jeff De Luca



Peter Coad

FDD



Pojawiło się przez manifestem



Podjęcie dla większych zespołów (do 500)



Nacisk na tworzenie oprogramowania dobrej jakości



Częste dostarczanie działających elementów



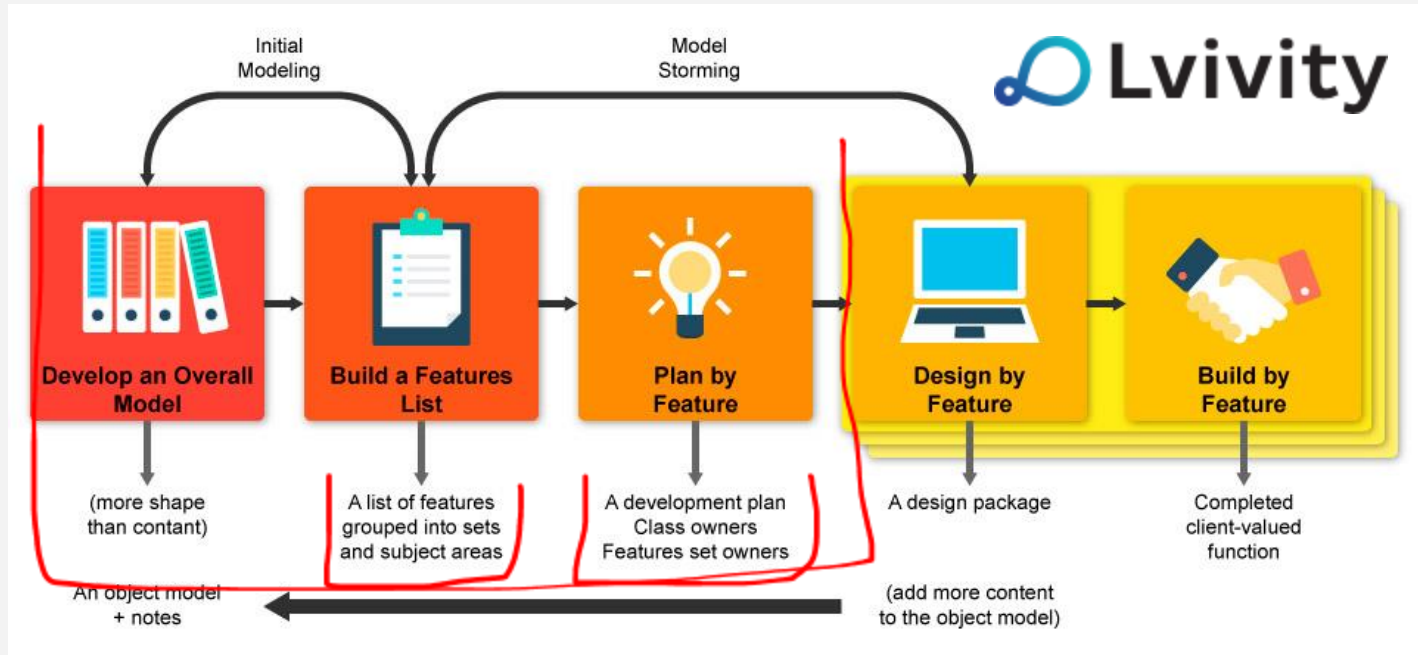
Włożenie na początku odpowiedniego wysiłku przed
rozpoczęciem iteracji



Jasny i oczywisty postęp oraz status projektu

FDD

<https://lvivity.com/7-things-about-feature-driven-development>



KANBAN

TO DO



DOING



DONE



 **SAFe® STUDIO**

06.08



Kanban / SAFe / LeSS

KANBAN

Kanban Guide for Scrum Teams



Wizualizacja pracy



Limit prac w toku (WIP): poprawia przepływ i ogranicza wąskie gardła



Płynny przepływ pracy



Scrum + Kanban = Scrumban

What Is A Kanban Board? The Ultimate Guide



SAFe: Scaled Agile Framework



Agile dla dużych: wiele zespołów (50+) pracuje nad jednym produktem



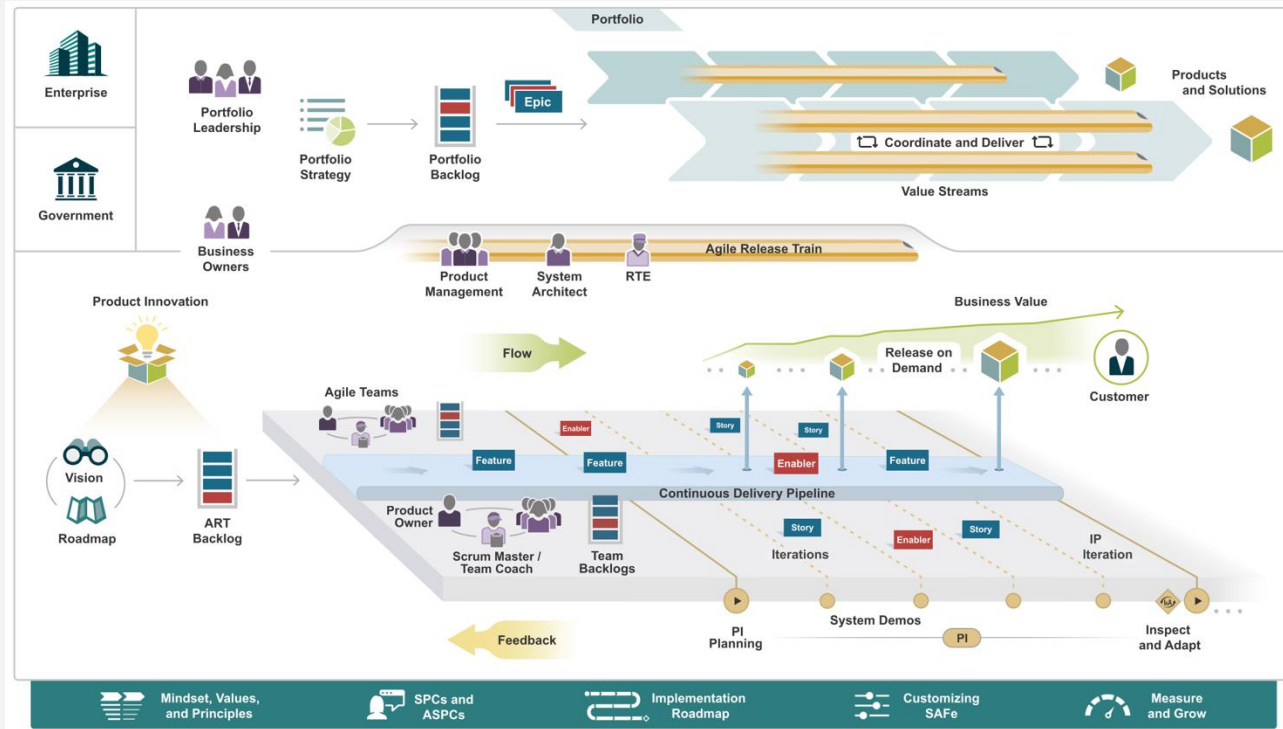
Poziomy planowania



Planning Interval (PI): 8-12 tygodni timebox; planowanie i synchronizacja



SAFe: Scaled Agile Framework



LeSS: Large-Scale Scrum



Skalowanie Scrum: 2-8 zespołów LeSS Basic; 8+ LeSS Huge



Minimalizacja złożoności procesów

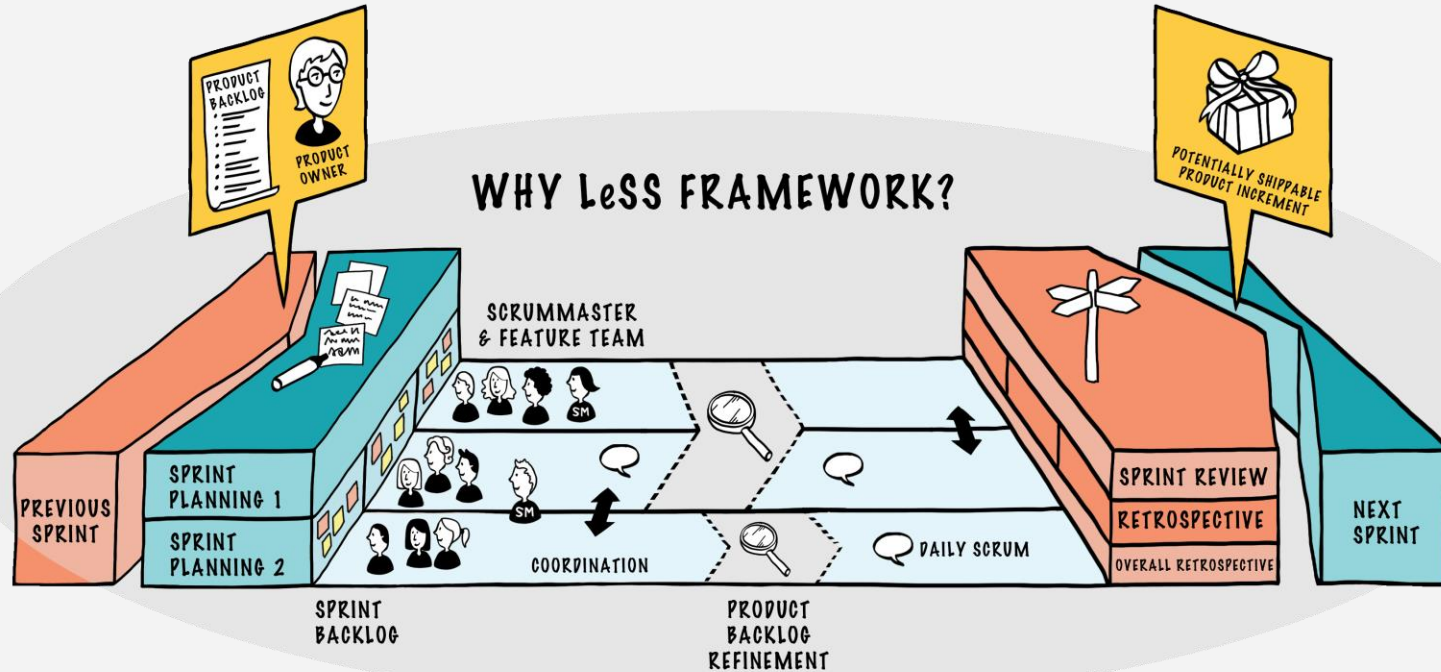


Maksymalizacja uczenia się i adaptacji



LeSS

<https://less.works/less/framework>





Koniec!